

Warehouse Safety Monitoring System

Master Project Status Report

Cross-Functional Engineering Synthesis — Computer Vision, Rule Engine, LLM Reporting & Backend Integration

Report Date	August 6, 2026
Prepared For	AI Builders Iran — Steering Committee / Project Leadership
Prepared By	Office of the Lead Systems Analyst
Sources Consolidated	LLM Reporting Module Technical Review (Aug 3, 2026); Backend Engineering Review — FastAPI/Gradio/Containerization (Aug 6, 2026); Video Processing & LLM Integration Layer Work Report; Warehouse Safety Rule Engine Documentation (v2.0)
Classification	Internal — Engineering & Steering Committee Distribution

Table of Contents

- 1. Executive Summary & Strategic Alignment 3
 - 1.1 Project Vision & Strategic Context 3
 - 1.2 Consolidated KPI Dashboard..... 3
 - 1.3 Major Milestones Achieved to Date 4
- 2. Cross-Functional Component Analysis..... 4
 - 2.1 Computer Vision & Rule Engine Layer 4
 - 2.2 LLM Reporting Layer 6
 - 2.3 Integration & Backend Layer..... 7
 - 2.4 Cross-Cutting Integration Finding 7
- 3. Issues, Technical Debt & Risk Assessment Register..... 8
 - 3.1 Identified Technical Debt 8
 - 3.2 Formal Risk Assessment Matrix 8
- 4. Financials, Operations & Strategic Roadmap..... 9
 - 4.1 Resource Capacity & Budget Consumption 9
 - 4.2 Immediate Tactical Action Items — Next Reporting Period 10
 - 4.3 Long-Term Milestone Projection 11
 - 4.4 Executive Recommendations for Steering Committee Review 11

1. Executive Summary & Strategic Alignment

1.1 Project Vision & Strategic Context

The Warehouse Safety Monitoring System is a flagship product initiative under AI Builders Iran's product-focused engineering program. Its purpose is to demonstrate a complete, deployable AI pipeline that converts raw warehouse video, without manual intervention at any stage, into structured safety-violation data and then into natural-language Health, Safety & Environment (HSE) reports suitable for both floor-level operations staff and executive leadership.

The system integrates three previously independent engineering workstreams behind a single service boundary: a computer-vision detection and multi-object tracking layer, a configurable rule-based safety-violation engine, and a locally hosted large language model (Qwen2.5-1.5B-Instruct) responsible for report generation in both Persian and English. These are orchestrated by a FastAPI backend and consumed through a lightweight Gradio client, exposed to end users as a single upload-and-analyze workflow.

Strategically, the project serves two purposes simultaneously: it is a customer-facing demo asset showcasing AI Builders Iran's ability to own a full pipeline from pixels to prose, and it establishes a reusable reference architecture — thin client / heavy service separation, environment-driven configuration, and once-per-process-lifetime model loading — that future product engagements can build on directly.

1.2 Consolidated KPI Dashboard

Dimension	Status	Commentary
Scope	ON TRACK	All four planned subsystems — detection/tracking, rule engine, LLM reporting, and backend/UI integration — have been delivered and independently reviewed. Scope has held to the original four-module architecture with no material expansion.
Timeline	AT RISK	The LLM module review (Aug 3) identified two blocking gaps — an empty pipeline entry point and an outdated data schema — that must close before the module is demo-ready. The backend review (Aug 6) subsequently surfaced two additional High-severity items gating any staging deployment.
Budget	NOT REPORTED	None of the three submitted engineering reports included cost, compute-hour, or budget-consumption figures. This is treated as a reporting gap rather than a favorable or unfavorable variance; see Section 4.1.
Quality	STRONG	Reviewers across all three component reports independently characterized the core engineering — detection logic, rule evaluation, prompt construction, model inference, and service orchestration — as sound, well-documented, and production-conscious (deterministic decoding, defensive schema normalization, graceful degradation without a GPU).

Dimension	Status	Commentary
Overall Health	AMBER	The underlying architecture and component-level engineering are solid. The project's current risk is concentrated in a small number of already-diagnosed integration, security, and packaging items rather than open-ended technical uncertainty.

1.3 Major Milestones Achieved to Date

- Detection and tracking module (YOLODetector) implemented on top of Ultralytics' native tracking API, with per-person hardhat/vest (PPE) association and per-object duration and movement computation.
- Rule engine reached version 2.0, shipping eight independently configurable safety rules, full Google-style docstrings, type hints across all functions, and duplicate-alert suppression.
- A latent rule-engine startup defect — an unintended Python string concatenation between a stray docstring and a configuration key — was diagnosed and corrected, restoring correct distance-threshold initialization.
- The SafetyVideoPipeline wrapper, unifying detection and rule evaluation behind a single process_video() call, was implemented and unit-verified against mocked dependencies and a synthetic critical-severity scenario (machinery-person proximity).
- The LLM reporting layer's model-loading and prompt-construction logic (llm_client.py, prompt.py) was independently reviewed and confirmed usable as-is, including deterministic, reproducible decoding appropriate for compliance-grade reporting.
- A corrected Pydantic schema (models_v2.py) was produced, reconciling the LLM module's expected input shape with the pipeline's actual video_info / tracking_summary / alerts_report JSON output.
- A two-service backend — a FastAPI orchestration layer and a Gradio client — was delivered, wiring all three modules behind a single POST /analyze endpoint with startup-once model loading and graceful degradation when no GPU is available.

2. Cross-Functional Component Analysis

This section provides an exhaustive, sub-team-by-sub-team evaluation of every component report submitted for this cycle, together with the technical integration points and API/data dependencies that connect them.

2.1 Computer Vision & Rule Engine Layer

The detection and tracking stage (YOLODetector) loads the trained YOLO model once per service lifetime rather than per video, runs detection and multi-object tracking on every frame via Ultralytics' .track() API with configurable confidence and IoU thresholds, and determines PPE compliance per person through bounding-box overlap against hardhat and safety-vest detections. Frame-relative position and size (normalized to a 0–1 range) are attached to every detection so downstream rules can reason about zones independently of video

resolution, and each object's first/last-seen frame and trajectory are tracked to support duration- and speed-based rules.

The rule engine evaluates eight safety conditions per frame, listed below with their configured default severities:

Rule	Trigger Condition	Default Severity
Vehicle–person proximity	Person within unsafe distance of a vehicle	High
Machinery–person proximity	Machinery within unsafe distance of a person	Critical
Missing PPE (general)	Person with no PPE for N consecutive frames	High
Missing hardhat	Person with no hardhat for N consecutive frames	Medium
Missing vest	Person with no safety vest for N consecutive frames	Medium
Idle worker	Person stationary beyond a time threshold	Low
Crowd detection	Too many people in one frame/zone	Medium
Abnormal speed	Object moving faster than the configured threshold	Medium

All distance thresholds are calibrated in real-world meters and converted to pixels via a configurable pixels-per-meter factor, allowing the same rule set to be reused across cameras with different framing. The SafetyVideoPipeline wrapper resets rule-engine state at the start of every video (preventing cross-video alert leakage) and returns a single dictionary containing video_info, tracking_summary, and alerts_report — the exact shape the LLM prompt generator is designed to consume.

VERIFICATION PERFORMED

Check	Method	Result
Module syntax	py_compile	Passed
Component isolation	YOLODetector and SafetyVideoPipeline instantiated with mocked YOLO/OpenCV dependencies	Passed — classes build and wire together correctly
End-to-end logic	Synthetic frame data (person placed next to machinery) fed through the rule engine	Passed — correctly produced a machinery_person_proximity alert at critical severity

Full end-to-end validation against real warehouse footage and the production model weights remains outstanding; it was not possible in the review environment due to missing ultralytics and opencv-python

dependencies, though the detection logic was ported directly from the team's already-validated detection script, so no behavioral change is expected.

2.2 LLM Reporting Layer

Four files were submitted for review, forming the module's reporting layer:

File	Approx. Size	Role	Review Status
llm_client.py	~8 KB	Model download + text generation (Qwen2.5-1.5B-Instruct)	Sound, usable as-is
prompt.py	~8 KB	Builds the four report prompt templates	Sound, usable as-is
models.py	~4 KB	Pydantic input schema	Superseded — did not match pipeline output
llm_main.py	0 KB	Intended pipeline entry point	Empty — not implemented

llm_client.py wraps tokenizer and model loading onto a CUDA GPU in bfloat16 precision and exposes single-turn, chat-style generation. Decoding is deterministic (do_sample=False), which is the correct choice for a compliance-facing safety document: identical input data will always produce an identical report, which is valuable both for audit purposes and for rehearsing demo output in advance. There is no CPU fallback path — initialization raises immediately if no GPU is present.

prompt.py defines HSEPromptGenerator with four report builders — daily_report, weekly_report, incident_report, and executive_summary — each returning fixed Markdown section headers and each explicitly instructing the model to use only the supplied data and never invent or infer information, a sound guardrail for a safety-compliance artifact.

The most significant finding in this component was a structural mismatch between models.py's RuleEngineData schema (time / events / statistics / total_count) and the tracking pipeline's actual output shape (video_info / tracking_summary / alerts_report). As delivered, feeding real pipeline output into RuleEngineData.model_validate() would fail validation outright. A corrected schema, models_v2.py, was produced during the review, introducing dedicated VideoInfo, TrackingSummary, Alert, and AlertsReport models — including dynamically keyed dictionaries for per-object and per-class data, and optional machinery_id / distance_m fields that apply only to proximity-type alerts.

Separately, llm_main.py — the intended entry point tying model loading, data loading, and report generation together — was found to be empty. A reference implementation and a full setup runbook were supplied as part of the review deliverables so the Demo Team has a concrete, working path from a tracking JSON file to a rendered report.

2.3 Integration & Backend Layer

The backend follows a thin-client / heavy-service pattern: the Gradio UI never imports model code and communicates with the API purely over HTTP, which lets the two services be deployed, scaled, and versioned independently — for example, a CPU-only UI container calling a GPU-backed API container. The request flow runs as follows: the UI posts a video plus language/style parameters to the API; the API persists the upload and passes it to `SafetyVideoPipeline.process_video()`; the resulting alerts are normalized and passed into either `HSEPromptGenerator` or a custom-prompt `AlertsSummarizer`; the assembled prompt is sent to the locally hosted Qwen2.5 model; and the generated report is returned to the UI alongside the underlying structured data.

Endpoint	Key Parameters	Behavior
GET /health	—	Liveness probe; reports whether the YOLO pipeline and LLM loaded successfully, with error detail if not.
POST /analyze	video, language (fa/en), mode, custom_prompt, max_new_tokens	Runs the full pipeline and returns video_info, tracking_summary, alerts_report, report_text, and the exact prompt used.

The backend review identified six strengths worth preserving as house practice: a clean HTTP-only service boundary; startup-once loading of the YOLO pipeline and LLM inside a FastAPI lifespan handler; graceful degradation when CUDA is unavailable (the API stays up and returns a clear 503 only on LLM-dependent calls); an explicit, well-commented normalization shim bridging a real data-contract gap; fully environment-overridable configuration (model paths, thresholds, upload limits, token bounds); and request-level validation that rejects malformed input with a 422 before any expensive work begins.

The single most consequential backend finding concerns `_normalize_alerts_for_summarizer()`: per the in-code documentation, `AlertsSummarizer` unconditionally reads `track_id`, `class_name`, and `speed` from every alert, but the rule engine only attaches `class_name` and `speed` to `high_speed` alerts, and crowd-detection alerts carry no `track_id` at all. Without the API's defensive normalization step, any realistic alert set would crash prompt generation with a `KeyError`. The fix is functionally correct but is, by the reviewer's own characterization, a temporary shim rather than a permanent solution.

2.4 Cross-Cutting Integration Finding

A Systemic Pattern, Not Two Isolated Bugs

Two independent reviews, conducted three days apart, each discovered a real mismatch between the rule engine's alert output and what a downstream LLM-facing consumer expected: `RuleEngineData / Event / Statistics` failed to match `video_info / tracking_summary / alerts_report` in the August 3 LLM-module review, and `AlertsSummarizer`'s unconditional field reads failed to match the rule engine's actual per-alert-type field set in the August 6 backend review. Both were patched locally — `models_v2.py` in one case, `_normalize_alerts_for_summarizer()` in the other — rather than fixed at the source. This recurrence across two unrelated integration boundaries, discovered independently by two different

reviewers, indicates a systemic gap rather than incidental bugs: the project currently has no single, versioned, canonical definition of "what an alert looks like" that all three sub-teams build against. This is elevated to the top item of the Risk Register in Section 3.2.

3. Issues, Technical Debt & Risk Assessment Register

3.1 Identified Technical Debt

The following items represent accumulated technical debt across the three reviewed components. None are currently blocking a code walkthrough or internal demo, but each increases the cost or risk of a shared/staging deployment the longer it is left unaddressed.

- Two independent, undocumented-as-permanent normalization shims (models_v2.py's schema reconciliation and the API's `_normalize_alerts_for_summarizer()`) currently stand in for a single canonical, versioned alert schema shared across the CV, rule-engine, and LLM sub-teams.
- Both delivered Dockerfiles build on python:3.14-slim with fully unpinned dependencies (fastapi, torch, ultralytics, gradio, and others), so build reproducibility is not currently guaranteed.
- A docker-compose.yml is referenced throughout comments and code (build context, API_URL service networking, GPU passthrough) but was not included in the delivered archive, so the two services cannot currently be run as documented.
- No requirements.txt, setup.py, or `__init__.py` was included with the LLM module archive, and no automated test suite (unit or integration) accompanies either the LLM module or the backend service.
- An empty, apparently unused Dockerfile exists at the project root alongside the two populated per-service Dockerfiles, creating ambiguity for the next engineer who builds the image.
- The LLM inference path has a hard CUDA requirement with no CPU or quantized fallback, which is an accepted trade-off for now but a single point of failure for any demo run on unverified hardware.

3.2 Formal Risk Assessment Matrix

Risk	Likelihood	Severity	Impact	Mitigation	Owner
Shared alert-schema divergence between the rule engine and its LLM-facing consumers (recurring across two integration points)	High	High	Report-generation crashes (KeyError); rework cost each time a consumer is added; demo failure risk	Convene a joint CV/LLM schema-alignment session; publish one canonical, versioned alert contract; retire both shims once adopted	CV + LLM sub-teams

Risk	Likelihood	Severity	Impact	Mitigation	Owner
Open CORS policy (allow_origins="*") and no authentication on /analyze	Medium	High	Any reachable origin can consume GPU inference time or exfiltrate uploaded video/report content	Restrict allow_origins to known UI origin(s); add at minimum an API-key or internal-network restriction before shared deployment	Backend team
Hard CUDA dependency with no CPU or quantized fallback	Medium	Medium	Live demo failure if run on unverified or GPU-less hardware	Confirm GPU/CUDA availability ahead of any demo; evaluate a documented quantized or CPU fallback path	LLM sub-team
Missing docker-compose.yml / non-reproducible multi-container build	High	Medium	Delays staging deployment; two Dockerfiles cannot currently be run as documented	Include a compose file (or build/run script) in the next delivery, with GPU reservation for the API service	Backend team
Unpinned dependencies on a bleeding-edge Python 3.14 base image	Medium	Medium	Build failures if GPU-critical wheels (torch, ultralytics) are unavailable for 3.14; non-reproducible images	Pin major/minor versions of GPU-critical packages; verify wheel availability, or step down to Python 3.11/3.12	Backend team
No automated test coverage on either the LLM module or the backend	High	Low–Medium	Silent regressions as the alert schema or prompt contract evolves	Start with a unit test around the alert-normalization helper, given it encodes an important cross-team assumption	Backend team
Full end-to-end validation with real footage and production model weights still pending	Medium	Medium	Real-world detection accuracy and alert precision remain unconfirmed ahead of any external demo	Schedule an integration test run against real warehouse footage with production weights before the next milestone	CV / LLM sub-teams

4. Financials, Operations & Strategic Roadmap

4.1 Resource Capacity & Budget Consumption

Three sub-teams are visible across the submitted component reports: a computer-vision / rule-engine sub-team, an LLM sub-team, and a backend team responsible for the FastAPI/Gradio integration layer. All three share a single hard infrastructure dependency — an NVIDIA GPU with CUDA support — since both the API container's LLM inference step and any full pipeline run require it; the UI container and vision-only API functionality remain usable without one.

Reporting Gap: No Budget or Compute-Cost Data Supplied

None of the three submitted engineering reports contained cost, cloud/compute-hour, or budget-consumption figures. In the interest of accuracy, this report does not estimate or fabricate financial figures on their behalf. It is recommended that the Project Management Office request standardized cost and compute-utilization reporting (GPU-hours consumed, cloud spend if applicable, engineering hours by sub-team) be included in each sub-team's report for the next cycle, so the steering committee has full financial visibility alongside the strong technical visibility already provided.

4.2 Immediate Tactical Action Items — Next Reporting Period

#	Action Item	Owner	Priority
1	Ship a functioning llm_main.py as a permanent module, replacing the ad hoc reference script supplied during review	LLM sub-team	High
2	Replace models.py with models_v2.py and update prompt.py's four type hints from RuleEngineData to PipelineReport	LLM sub-team	High
3	Add a requirements.txt (pydantic, torch, transformers, huggingface_hub) for the LLM module	LLM sub-team	Medium
4	Convene a joint CV + LLM + Backend schema-alignment session to unify the alert contract and retire both normalization shims	Project Lead / All sub-teams	High
5	Lock down CORS to known origins and add access control (API key or internal-network restriction) to /analyze	Backend team	High
6	Add the missing docker-compose.yml, including a GPU reservation for the API service	Backend team	Medium
7	Pin GPU-critical dependency versions and confirm wheel availability for the chosen Python base image, or step down to 3.11/3.12	Backend team	Medium
8	Remove or repurpose the empty root-level Dockerfile	Backend team	Low
9	Add a unit test around the alert-normalization helper as the first entry in an automated test suite	Backend team	Medium
10	Schedule full end-to-end validation against real warehouse footage with production model weights	CV / LLM sub-teams	High

4.3 Long-Term Milestone Projection

Near-term (next cycle): resolve the two High-severity backend findings and the LLM module's entry-point/schema gap to reach a fully demo-ready state.

Mid-term: staging deployment enabled by CORS/auth lockdown, a compose-based multi-container deployment with GPU reservation, and pinned, reproducible dependencies.

Longer-term: production hardening — a documented CPU or quantized inference fallback, a growing automated test suite anchored on the alert-normalization contract, monitoring and logging for processing time and alert volume, and formal cost/compute-utilization tracking folded into standard reporting.

4.4 Executive Recommendations for Steering Committee Review

Recommended Steering Committee Actions

(1) Treat the two independently discovered alert-schema mismatches as a single systemic finding requiring one canonical, versioned data contract jointly owned by the CV, rule-engine, and LLM sub-teams — not two separate local patches. (2) Gate any staging or shared-environment deployment behind resolution of the two High-severity backend findings: open CORS/no authentication, and the missing docker-compose.yml. (3) Direct that formal budget and compute-cost reporting be added to each sub-team's report starting next cycle, closing the financial-visibility gap noted in Section 4.1. (4) Approve the entry-point and schema-correction work already scoped in the LLM module review (models_v2.py and the reference llm_main.py) as unblocking work for the next demo milestone.